Управление проектом и движком через CMake

Применимо к FemboyEngine2D 0.1.0-alpha.1 и CMake 3.28 или новее.

Требования к инструментам

Движку нужен C++26:

- Windows: clang-cl 19+ с MSVC STL или MSVC 19.40+.
- Linux: Clang 19+ или GCC 15+.
- macOS/iOS: Apple Clang с поддержкой C++26.
- Android: NDK r27+ с C++26-совместимым Clang.

Рекомендуемый генератор — Ninja. Генераторы Visual Studio поддерживаются, но сборка этого дерева с ними медленнее.

Настройка отдельной сборки

```
cmake -S . -B build -G Ninja \
  -DCMAKE_BUILD_TYPE=RelWithDebInfo \
  -DCMAKE_C_COMPILER=clang-cl \
  -DCMAKE_CXX_COMPILER=clang-cl
cmake --build build --parallel 8
ctest --test-dir build --output-on-failure
```

Для тестов используйте `RelWithDebInfo`. Debug-сборка допустима, но часть GPU- и video-тестов становится на порядок медленнее и может упираться в таймауты CI.

Опции движка

Опция	По умолчанию	Назначение
`FBY_BUILD_EDITOR`	`ON`	Библиотека и приложение редактора на Dear ImGui.
`FBY_BUILD_SANDBOX`	`ON`	Исполняемый файл sandbox.
`FBY_BUILD_TESTS`	`OFF`	Сборка набора CTest.
`FBY_BUILD_TOOLS`	`ON`	`fby_fempak`, `fby_aotc`, `fby_fontc`, `fby_scene_tool`.
`FBY_BUILD_PLATFORM_PROVIDERS`	`ON`	Общие библиотеки провайдеров Steam/EGS/GOG.
`FBY_RHI_BACKEND`	`auto`	`auto`, `vulkan`, `metal` или

		`null`.

Движок собирается без Vulkan SDK: поддерживаются `FBY_RHI_BACKEND=null` и null-RHI для headless CI. Если заголовки Vulkan недоступны, редактор собирается с минимальными Khronos-заголовками, которые поставляет SDL.

Публичные алиасы CMake

Подключайте только PascalCase-алиасы. Это поддерживаемый API сборочного дерева.

Алиас	Назначение
`FemboyEngine2D::Engine`	Runtime-библиотека. Её подключает игра.
`FemboyEngine2D::Editor`	Библиотека редактора и UI-стек.
`FemboyEngine2D::AssetCook`	Кукинг текстур BC7/ASTC.
`FemboyEngine2D::AssetLib`	Компилятор паков и локализации.
`FemboyEngine2D::ShaderCompilerCore`	Встроенный компилятор GLSL/SPIR-V.

Внутренние цели (`fby_engine`, `fby_editor`, `fby_fempak`, `fby_aotc`) и имена исполняемых файлов — детали реализации. Они не переименовываются ради стиля и не являются публичным контрактом. Алиасов `FemboyEngine::*` и `FemboyEngine2D::Runtime` больше нет.

Подключение через add_subdirectory

```
cmake_minimum_required(VERSION 3.28)
project(MyGame LANGUAGES CXX)

set(FBY_ENGINE_ROOT "${CMAKE_CURRENT_SOURCE_DIR}/../.." CACHE PATH
"FemboyEngine2D root")
set(FBY_BUILD_EDITOR OFF CACHE BOOL "" FORCE)
set(FBY_BUILD_SANDBOX OFF CACHE BOOL "" FORCE)
set(FBY_BUILD_TESTS OFF CACHE BOOL "" FORCE)
set(FBY_BUILD_TOOLS OFF CACHE BOOL "" FORCE)
set(FBY_BUILD_PLATFORM_PROVIDERS OFF CACHE BOOL "" FORCE)

add_subdirectory("${FBY_ENGINE_ROOT}" "${CMAKE_BINARY_DIR}/_fby_engine"
EXCLUDE_FROM_ALL)

add_executable(MyGame GameMain.cpp)
target_link_libraries(MyGame PRIVATE FemboyEngine2D::Engine)
target_compile_definitions(MyGame PRIVATE
    FBY_PROJECT_ASSETS="${CMAKE_CURRENT_SOURCE_DIR}/assets")
```

Каталог сборки должен быть уникальным. Нельзя добавлять движок в тот же binary-dir, что и игру: CMake отклонит совпадающие имена целей.

`FBY_PROJECT_ASSETS` — это compile definition, а не cache-переменная движка. Сгенерированный `GameMain.cpp` использует её для монтирования каталога ассетов проекта.

Сгенерированный Clean Slate проект

`CreateProject()` создаёт:

```
MyGame/
  CMakeLists.txt
  project.femproj
  GameMain.cpp
  assets/
```

Сгенерированный CMakeLists отключает редактор, sandbox, тесты, инструменты и провайдеров магазинов. Он подключает только `FemboyEngine2D::Engine` и задаёт `FBY_PROJECT_ASSETS`. Сборка:

```
cmake -S MyGame -B MyGame/build -G Ninja -DCMAKE_BUILD_TYPE=RelWithDebInfo
cmake --build MyGame/build --parallel 8
```

Корень движка передаётся явно, потому что проект по умолчанию лежит в `<engine>/Projects/<name>`. Сгенерированный CMake также принимает переменную окружения `FBY_ENGINE_ROOT`; это сохраняет переносимость проекта между машинами и ОС:

```
export FBY_ENGINE_ROOT=/path/to/FemboyEngine2D
cmake -S MyGame -B MyGame/build
```

Абсолютный путь к движку не хранится в `project.femproj`. Пути ассетов внутри проекта относительные; абсолютный `FBY_PROJECT_ASSETS` вычисляется при конфигурации, поэтому перенос проекта требует переконфигурации/пересборки, а не миграции данных.

Явное значение cache по-прежнему имеет приоритет:

```
cmake -S MyGame -B MyGame/build -DFBY_ENGINE_ROOT=/path/to/FemboyEngine2D
```

Артефакты сборки

Цель	Выход
`fby_editor_app`	`build/bin/fby_editor`
`fby_sandbox`	`build/bin/fby_sandbox`
`fby_fempak`	`build/bin/fby_fempak`

`fby_aotc`	`build/bin/fby_aotc`
`fby_fontc`	`build/bin/fby_fontc`
`fby_scene_tool`	`build/bin/fby_scene_tool`

Тесты и style gate

```
cmake -S . -B build -G Ninja -DCMAKE_BUILD_TYPE=RelWithDebInfo
-DFBY_BUILD_TESTS=ON
cmake --build build --parallel 8
ctest --test-dir build --output-on-failure
python Source/Tools/Dev/CheckStyle.py
```

38 тестов — самостоятельные исполняемые файлы без фреймворка. Они работают headless и запускаются по отдельности из `build/bin`.

Выбор RHI

```
cmake -S . -B build -DFBY_RHI_BACKEND=vulkan
cmake -S . -B build -DFBY_RHI_BACKEND=null
```

`null` — настоящий backend с детерминированным учётом кадров, а не заглушка. Его используют CI и сгенерированный проект без GPU. `auto` выбирает Metal на Apple и Vulkan на Windows, Linux и Android при наличии заголовков.

Добавление цели

1. Добавьте исходники в `Source/Engine/CMakeLists.txt` или CMakeLists подсистемы.
2. Подключайте `FemboyEngine2D::Engine`, а не `fby_engine`.
3. Публичные include-каталоги оформляйте через `$<BUILD_INTERFACE:...>`.
4. Добавьте тест через `fby_add_engine_test()` в `Source/Tests/CMakeLists.txt`.
5. Запустите style gate и CTest.

Типичные ошибки

- Не найден `cmake/EmbedShaders.cmake`: движок подключён из игры, и `CMAKE_SOURCE_DIR` указывает на игру. Используйте штатный `add_subdirectory`; движок сам выводит свой корень из `CMAKE_CURRENT_SOURCE_DIR`.
- Нет `vulkan/vulkan.h`: настройте `-DFBY_RHI_BACKEND=null` или установите Vulkan-Headers. У редактора есть fallback на Khronos-заголовки SDL.
- Ошибки legacy-алиасов: замените `FemboyEngine::*` и `FemboyEngine2D::Runtime` на `FemboyEngine2D::Engine`.

- Тесты упираются в таймаут: используйте `RelWithDebInfo`, а не `Debug`.
- Ошибки C++26: обновите компилятор. C++23 не поддерживается.